Practical 10 Case study on UML diagram

Aim: represent the users and main functions of the assigned system using UML. **Requirements:** project brief, source code and database schema.

Case study: Visitors search and view schemes. Registered users maintain an optional profile, request assessments and review saved results. Administrators maintain schemes, categories, rules, documents and sources.

Use-case diagram

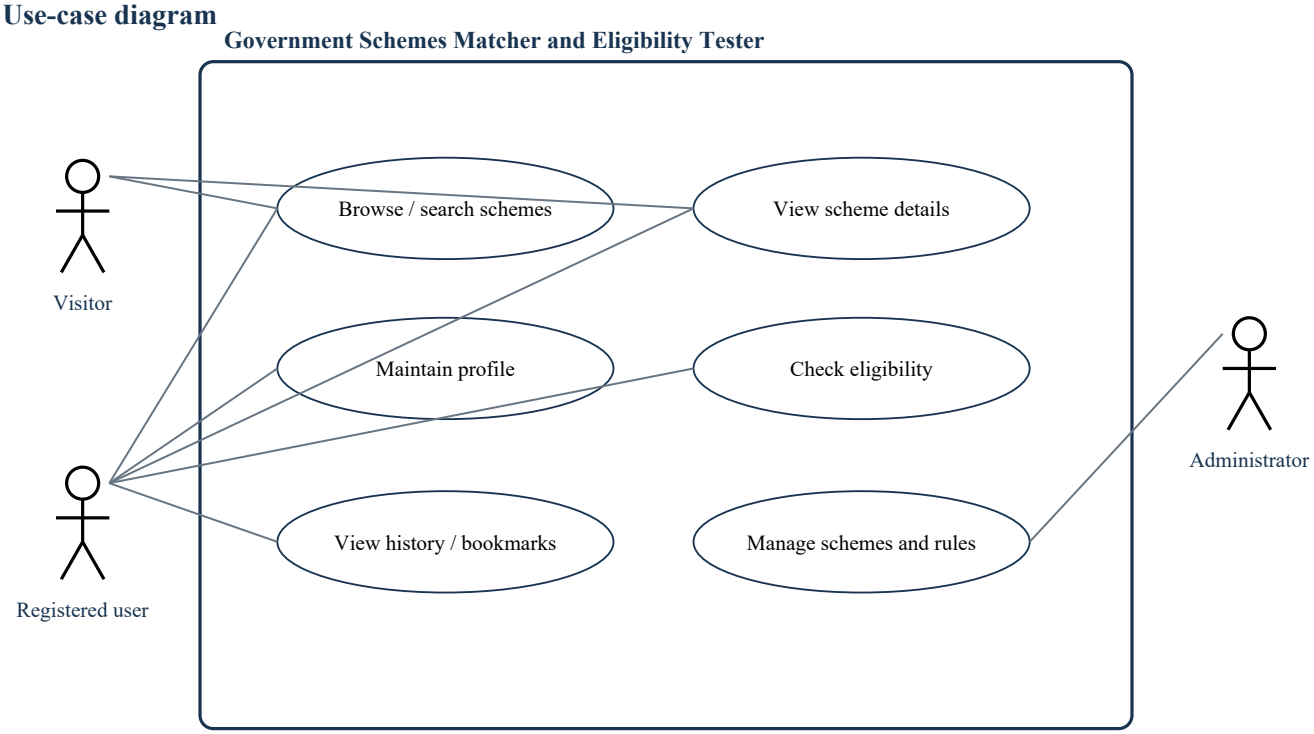


Figure 10.1. System boundary, actors and principal use cases.

Observation: the user-facing assessment is preliminary. The application must distinguish a failed condition from missing information and must restrict scheme maintenance to administrators.

Result: the use-case model identifies three actors and their main responsibilities.

Practical 10 Domain model and interaction

Domain / class view

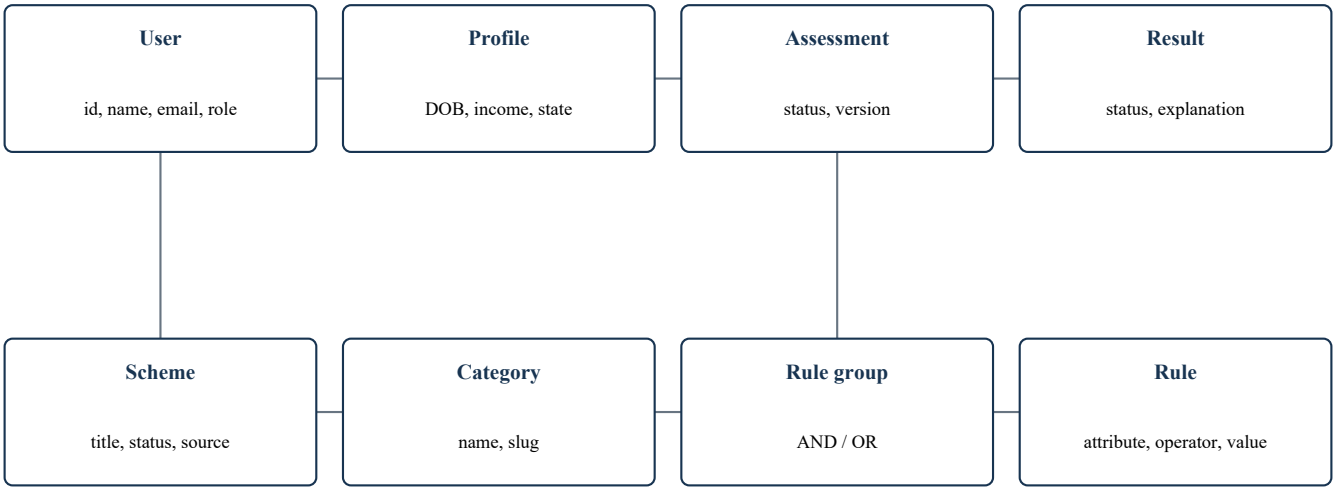


Figure 10.2. Core records and principal associations.

Association	Multiplicity / purpose
User - profile	One user has zero or one profile.
Scheme - category	Schemes are grouped into catalogue categories.
Scheme - rule group - rule	A scheme contains groups; each group contains typed rules.
User / scheme - assessment	Each assessment is owned by a user and refers to a scheme.
Assessment - result	An assessment stores per-condition outcomes and explanations.

Assessment sequence

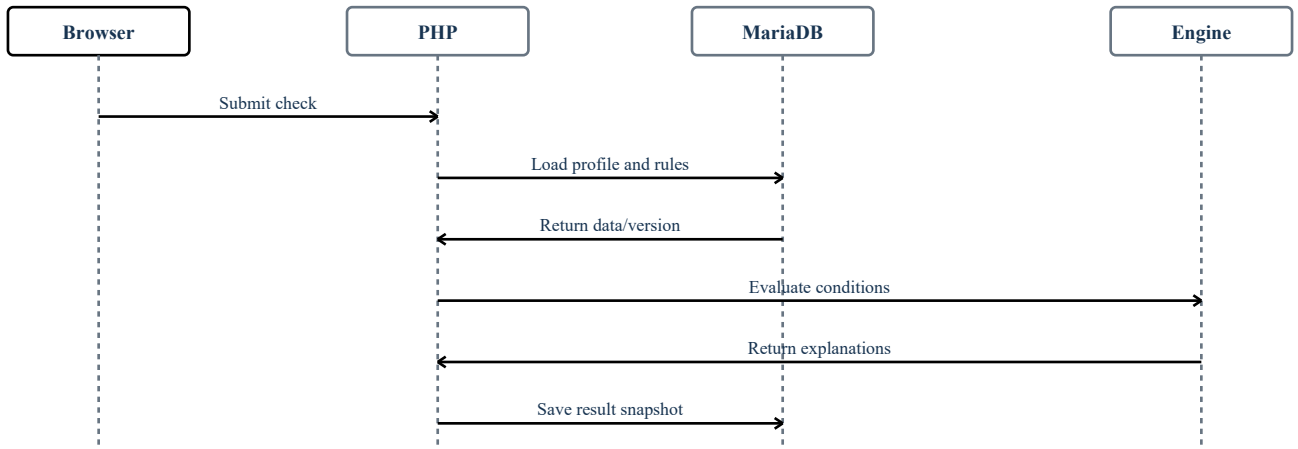


Figure 10.3. Browser, PHP, database and engine interactions.

Conclusion: these models explain how the controller connects account checks, stored rules, eligibility evaluation and saved results.

Practical 11 Code-level testing

Aim: verify PHP syntax and test the eligibility engine with controlled inputs. **Setup:** PHP 8.2.12; tests in **tests/Unit/EligibilityEngineTest.php**; runner **tests/run_unit.php**.

Method: lint the PHP source files, then call the engine with test profiles and rule groups. Compare each returned status and overall assessment with the expected result.

Unit-test cases 1-9

ID	Condition / input	Expected result	Run
UT01	Income equals maximum	PASS	Passed
UT02	Income exceeds maximum	FAIL / conditions not met	Passed
UT03	Required state is missing	UNKNOWN / more information	Passed
UT04	One rule fails in AND group	Conditions not met	Passed
UT05	One rule passes in OR group	Potentially eligible	Passed
UT06	OR group has unknown inputs only	More information needed	Passed
UT07	Age equals lower inclusive bound	PASS	Passed
UT08	State matches an allowed list value	PASS	Passed
UT09	Unsupported operator is supplied	INVALID / unable to assess	Passed

Evidence: syntax validation passed for 35 PHP files. The project runner executed the full 18-case suite with no failures.

Result: the selected boundary, grouping and missing-data tests passed.

Practical 11 Code-level testing (continued)

Unit-test cases 10-18

ID	Condition / input	Expected result	Run
UT10	Optional-only failed rule group	Not reported as eligible	Passed
UT11	Condition applicability is false	NOT APPLICABLE recorded	Passed
UT12	Malformed group operator	Unable to assess	Passed
UT13	Empty rule group	Unable to assess	Passed
UT14	Alternative group matches	Potentially eligible	Passed
UT15	Numeric operator on state text	Definition rejected	Passed
UT16	Age derived from date of birth	PASS	Passed
UT17	Missing value with exists checks	exists FAIL; not_exists PASS	Passed
UT18	Lowercase value in allowed list	PASS	Passed

Test run	Result
PHP lint	35 files checked; 0 syntax errors.
EligibilityEngine unit runner	18 passed; 0 failed.
PHPUnit	Not run; Composer was unavailable. The included PHP runner was used.

Conclusion: code-level tests support the eligibility engine's handling of boundaries, logical groups, missing values and invalid rule definitions. They do not replace database integration or security testing.

Limit: database and full browser integration are not part of this unit suite.

Practical 12 Manual testing

Aim: check the main user-facing workflows in the local browser. **Setup:** PHP application, isolated MariaDB demo database and fictional scheme records.

Procedure: perform each action in the browser and compare the visible response with the expected outcome. Account and profile values used for this check are demo data.

Executed cases

ID	Action	Expected outcome	Status
M01	Open home page	Home page and preliminary-only notice are visible.	Passed
M04	Sign in with demo admin account	Signed-in navigation and dashboard are available.	Passed
M06	Save profile information	Success message appears and entered values persist.	Passed
M07	Search catalogue by keyword	Matching fictional scheme is returned.	Passed

Observed: the home page displayed the independent-service notice. A demo administrator reached the signed-in application. Saving the profile returned “Profile saved.” The keyword “Student Learning” reduced the catalogue to the matching fictional education record.

Result: 4 of 4 selected manual cases passed in the local demo environment.

Browser evidence

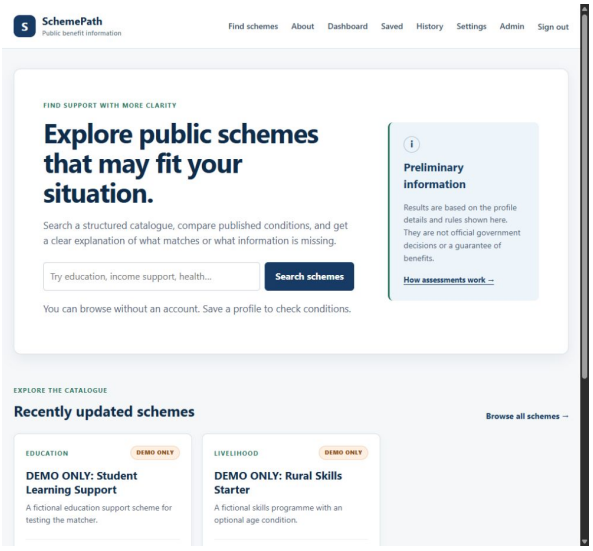


Figure 12.1. Home page with preliminary-use notice.

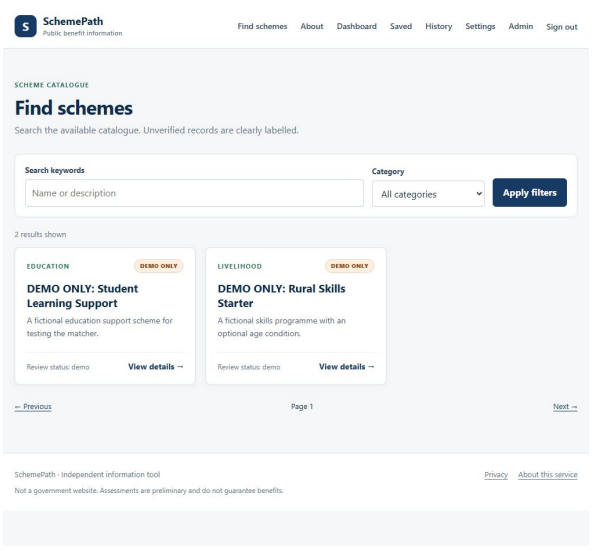


Figure 12.2. Catalogue with clearly labelled demo records.

Practical 12 Manual testing (continued)

Executed cases (continued)

ID	Action	Expected outcome	Status
M08	Filter by Education category	Only the Education demo scheme is listed.	Passed
M09	Open scheme details	Description, fictional-data label and review status appear.	Passed
M10	Check preliminary match	Rule outcomes and preliminary notice appear.	Passed

Observed: the Education filter returned one matching record. The details page showed the fictional-data label, benefits, application information, required document and demo review status. The assessment page showed two passed configured conditions, no unknown conditions, and the preliminary-only disclaimer.

Manual check summary	Outcome
Selected browser cases	7 passed; 0 failed; 0 pending in this selected record.
Environment	Local PHP and MariaDB demo setup; fictional scheme/profile data.
Coverage limit	Account recovery, security authorization, admin editing and invalid-form cases were not part of this selected run.

Conclusion: all seven recorded browser checks produced the expected visible result in the demo setup. This is a focused manual run and does not certify untested administrative or security workflows.

Assessment and administration screens

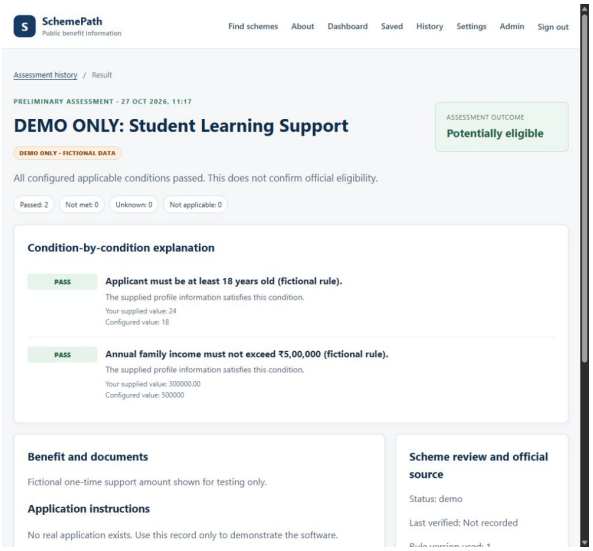


Figure 12.3. Preliminary result with two condition explanations.

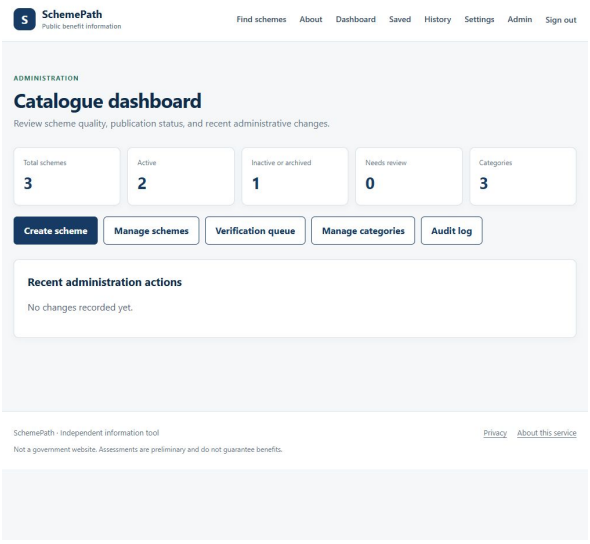


Figure 12.4. Local catalogue administration dashboard.

Practical 13 Implementation and evaluation

Aim: summarize the implementation and evaluate the assigned project against its main objective.

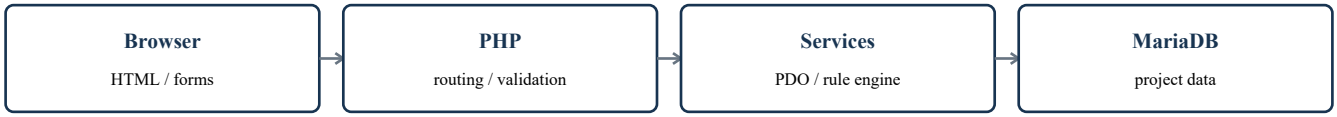


Figure 13.1. Application layers in the local XAMPP setup.

Module	Implementation
Catalogue	PHP views search and display scheme records, categories and review labels from MariaDB.
Accounts	Session authentication, password hashing, profile editing, bookmarks and owned history.
Administration	Protected pages maintain schemes, categories, rules, documents, sources and status.
Eligibility engine	Typed conditions return PASS, FAIL, UNKNOWN, NOT APPLICABLE or INVALID with explanations.

Evaluation: the prototype connects PHP to a relational database and returns explainable preliminary matches. It preserves missing information as UNKNOWN and saves the profile snapshot and rule version with an assessment.

Limitations: bundled schemes are fictional; email recovery and automated integration tests are absent; no production security audit has been performed. Real scheme data needs source verification before use.

Conclusion: the core educational objective is implemented. The unit tests passed, and seven selected browser workflows were observed in the local demo environment.